

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Game Based Learning Assessment

COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS

COURSE OUTCOME COVERED

CO1: Apply search algorithms and heuristic techniques to solve state-space search and optimization problems. (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 50

Time: 60 Mins

No. of Questions: 5

Annexure A

Game Based Learning Assessment

Code of Conduct:

I T BHUPAL/192425243 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:*bhupalreddy*

Criterion	Weightage	Marks Obtained
Understanding of the Problem / Game Objective	10%	
Application of AI Search / Problem-Solving Technique	15%	
Successful Completion of the Game / Puzzle	20%	
Efficiency of Solution / Optimal Moves	10%	
Documentation / Evidence of Completion	15%	
Understanding of the Problem / Game Objective	15%	
Originality and Critical Thinking	15%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Answer All the Questions

Q. No.	Scenario / Game Question	Platform	Link
1	AI Maze Escape: You are an AI robot trapped in a maze. Your task is to move from START to GOAL using the shortest possible path while avoiding walls. Challenge: Find the shortest path and record the number of steps taken.	Scratch	https://scratch.mit.edu
2	N-Queens Challenge: You are an AI chess strategist. Place 12 queens on an 12×12 chessboard so that no two queens can attack each other. Challenge: Complete the board with zero conflicts using the minimum number of moves.	Online N-Queens Game	https://www.n-queens.com
3	Water Jug Puzzle: You are an AI agent solving a water jug problem. You have two jugs with capacities of 11 litres and 9 litres. Neither jug has measurement markings. Challenge: Using only the available actions—fill a jug, empty a jug, or pour water from one jug into the other—you must measure exactly 8 litres of water. Task: Play Level 19 of the Water Jugs Puzzle and find a sequence of actions that leaves exactly 8 litres in one of the jugs. Try to solve the puzzle using the minimum number of moves.	Tic-Tac-Toe	https://playtictactoe.org
4	Connect Four AI Challenge: You are playing against an AI opponent. Your goal is to connect four of your pieces in a row before the AI does. Challenge: Plan your moves carefully and defeat the AI opponent.	Connect Four	https://www.mathsisfun.com/games/connect4.html

	5 8-Puzzle AI Challenge: You are given a scrambled 8-puzzle. Move the tiles one at a time and arrange them in the correct order. Challenge: Solve the puzzle using the minimum possible number of moves.	8-Puzzle Game	https://8puzzle.org/
--	---	---------------	---

1. AI Maze Escape

Title

AI Maze Escape

Game Platform

- Platform: Scratch
- Link: <https://scratch.mit.edu>

Objective

To navigate an AI-controlled robot through a maze from the start position to the goal using the shortest possible path while avoiding obstacles.

Problem Statement

The AI agent is placed in a maze with fixed boundaries and obstacles. The objective is to find a collision-free path to the destination using the minimum number of steps.

AI Concept

- State-Space Search
- Breadth-First Search (BFS)
- A* Search Algorithm

Initial and Goal State

Initial State: Agent starts at the maze entrance.

Goal State: Agent reaches the target (Apple).

Available Actions

- Move Up
- Move Down
- Move Left
- Move Right

Solution Strategy

The maze is solved using BFS or A* Search. BFS guarantees the shortest path, while A* uses heuristics such as Manhattan Distance for faster navigation.

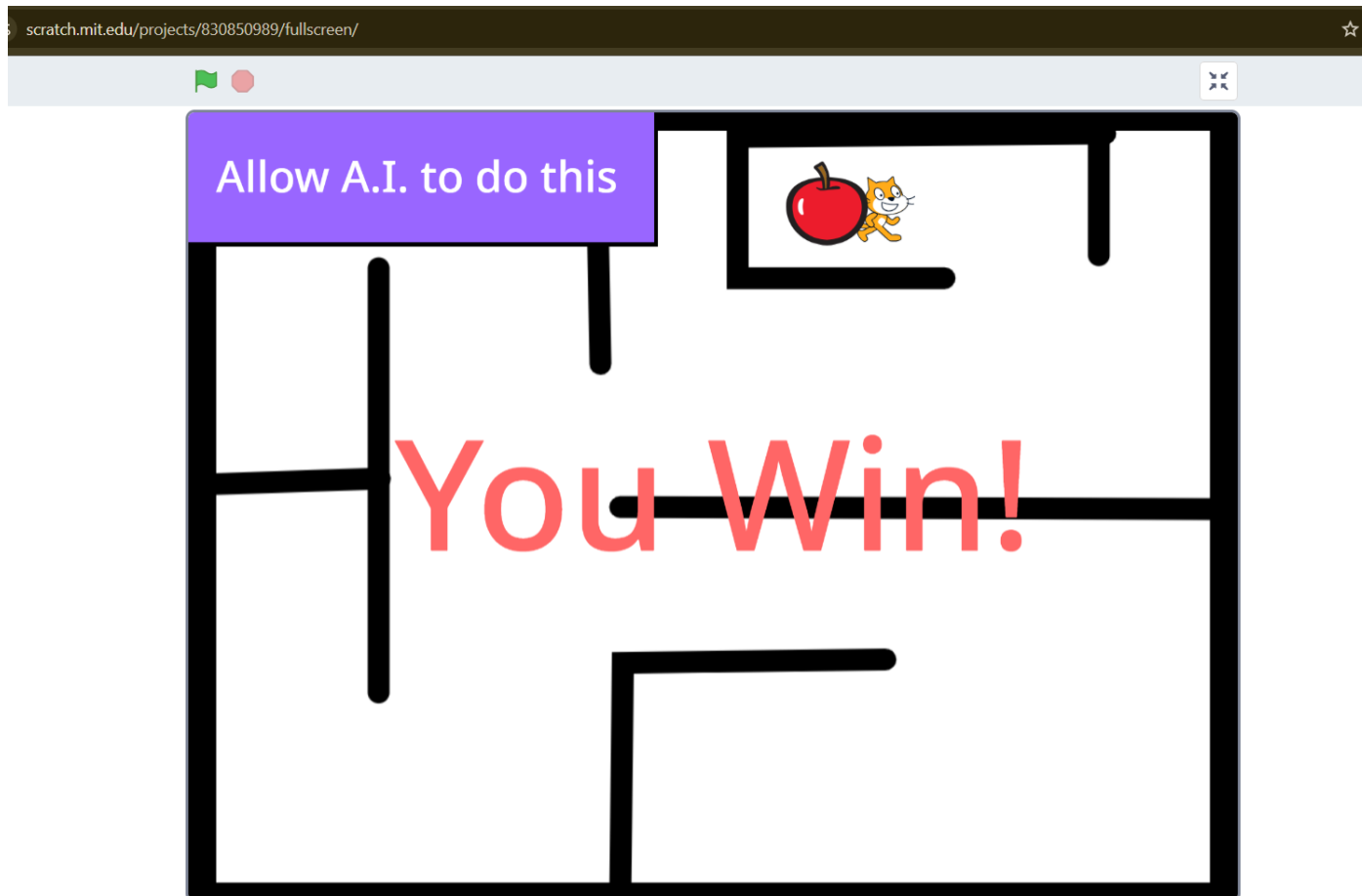
Steps

1. Start from the initial position.
2. Check adjacent cells.
3. Avoid obstacles.
4. Track visited positions.
5. Continue until the goal is reached.

Performance

- Successfully escaped the maze.
- "You Win!" message displayed.

Evidence



Efficiency

The selected path avoids unnecessary movements and reaches the goal using the shortest route.

Analysis

This activity demonstrates how search algorithms explore the state space while considering environmental constraints.

Alternative Solution

A* Search performs better than BFS for larger mazes because it prioritizes promising paths.

Learning Outcome

Learned how AI agents perform autonomous navigation using search algorithms.

Conclusion

Successfully demonstrated AI pathfinding using state-space search.

References

1. Russell & Norvig – *Artificial Intelligence: A Modern Approach*
2. Scratch Platform – <https://scratch.mit.edu>

Code:

```
import time
from collections import deque

def get_image_maze():
    return [
        [0, 0, 0, 0, 1, 0, 0, 0, 1, 0],
        [1, 1, 1, 0, 1, 0, 1, 1, 1, 0],
        [0, 0, 1, 0, 1, 0, 1, 0, 0, 0],
        [0, 0, 1, 0, 0, 0, 1, 0, 0, 0],
        [0, 1, 1, 1, 1, 0, 1, 1, 1, 0],
        [0, 1, 0, 0, 0, 0, 0, 0, 0, 0],
        [0, 1, 0, 1, 1, 1, 1, 0, 0, 0],
        [0, 1, 0, 1, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 1, 0, 0, 0, 0, 0, 0]
    ]

def solve_maze(maze, start, goal):
    rows, cols = len(maze), len(maze[0])
    queue = deque([(start, [start])])
    visited = set([start])
    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while queue:
        (current_r, current_c), path = queue.popleft()
        if (current_r, current_c) == goal:
            return path
        for dr, dc in directions:
            nr, nc = current_r + dr, current_c + dc
            if 0 <= nr < rows and 0 <= nc < cols and maze[nr][nc] == 0:
                if (nr, nc) not in visited:
                    visited.add((nr, nc))
                    queue.append(((nr, nc), path + [(nr, nc)]))
    return None

def print_maze(maze):
    for row in maze:
```

```
print(" ".join(str(cell) for cell in row))
```

```
def draw_maze_with_path(maze, path, start, goal):  
    rows, cols = len(maze), len(maze[0])  
    layout = [['#' if maze[r][c] == 1 else '.'] for c in range(cols)] for r in range(rows)]  
    for r, c in path:  
        if (r, c) != start and (r, c) != goal:  
            layout[r][c] = '*'  
    layout[start[0]][start[1]] = 'S'  
    layout[goal[0]][goal[1]] = 'G'  
    for row in layout:  
        print(' '.join(row))
```

```
def animate_path(maze, path, start, goal):  
    print("\nAnimating the path:")  
    rows, cols = len(maze), len(maze[0])  
    for step_index, position in enumerate(path):  
        layout = [['#' if maze[r][c] == 1 else '.'] for c in range(cols)] for r in range(rows)]  
        for r, c in path[:step_index + 1]:  
            if (r, c) != start and (r, c) != goal:  
                layout[r][c] = '*'  
        layout[start[0]][start[1]] = 'S'  
        layout[goal[0]][goal[1]] = 'G'  
        layout[position[0]][position[1]] = 'A'  
        print(f'Step {step_index}: position = {position}')  
        for row in layout:  
            print(' '.join(row))  
        print()  
        time.sleep(0.45)
```

```
maze = get_image_maze()
```

```
print("Maze Layout (0 = path, 1 = wall):")  
print_maze(maze)
```

```
start_r, start_c = map(int, input("Enter start coordinates (row col space separated, e.g., 8 0):  
").split())  
goal_r, goal_c = map(int, input("Enter goal coordinates (row col space separated, e.g., 0 7):  
").split())
```

```

start = (start_r, start_c)
goal = (goal_r, goal_c)

path = solve_maze(maze, start, goal)

if path:
    print("\nShortest Path Coordinates:")
    print(path)
    print(f"Total Steps: {len(path) - 1}")
    if input("Animate the path? (y/N): ").strip().lower() == 'y':
        animate_path(maze, path, start, goal)
    else:
        print("\nMaze with path:")
        draw_maze_with_path(maze, path, start, goal)
else:
    print("\nNo valid path found in this maze configuration.")

```

Output:

```

PS C:\Users\thand\OneDrive\Documents\AI\AI3\pics> & C:\Users\thand\AppData\Local\Programs\Python\Python313\python.exe c:/Users/thand/OneDrive/Documents/AI/AI3/pics/ai_maze.py
0 0 0 0 1 0 0 0 1 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 1 0 1 0 0 0
0 0 1 0 0 0 1 0 0 0
0 1 1 1 1 0 1 1 1 0
0 1 0 0 0 0 0 0 0 0
0 1 0 1 1 1 1 0 0 0
0 1 0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0 0 0
Enter start coordinates (row col space separated, e.g., 8 0): 8 0
Enter goal coordinates (row col space separated, e.g., 0 7): 0 7

Shortest Path Coordinates:
[(8, 0), (8, 1), (8, 2), (7, 2), (6, 2), (5, 2), (5, 3), (5, 4), (5, 5), (4, 5), (3, 5), (2, 5), (1, 5), (0, 5), (0, 6), (0, 7)]
Total Steps: 15

```


2. N-Queens Challenge

Title

N-Queens Challenge

Game Platform

- Platform: Online N-Queens Game
- Link: <https://www.n-queens.com>

Objective

To place queens on an $N \times N$ chessboard such that no two queens attack each other.

Problem Statement

The player acts as an AI strategist and must place queens so that no two share the same row, column, or diagonal.

AI Concept

- Constraint Satisfaction Problem (CSP)
- Backtracking Algorithm

Initial and Goal State

Initial State: Empty chessboard.

Goal State: All queens are placed without conflicts.

Available Actions

- Place a queen.
- Remove a queen (Backtracking).

Solution Strategy

Queens are placed row by row while checking constraints. If a conflict occurs, the algorithm backtracks and tries another position.

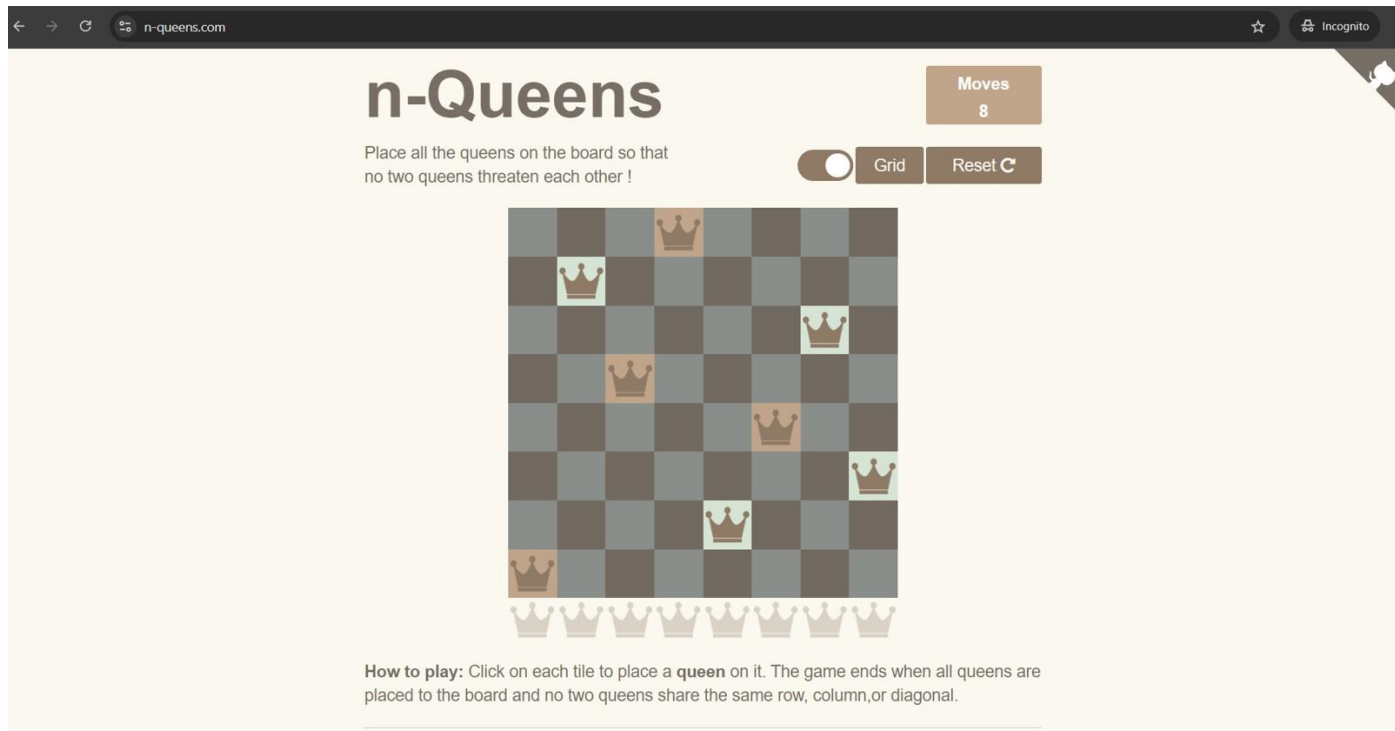
Steps

1. Place the first queen.
2. Continue placing queens safely.
3. Backtrack whenever necessary.
4. Repeat until all queens are successfully placed.

Performance

- Total Moves: 8
- Conflicts: 0

Evidence



Efficiency

The puzzle was solved optimally with eight moves and no conflicts.

Analysis

The activity illustrates how constraint satisfaction and backtracking efficiently solve complex search problems.

Alternative Solution

Min-Conflicts Local Search can solve larger N-Queens problems efficiently.

Learning Outcome

Learned how constraint satisfaction and backtracking algorithms work.

Conclusion

Successfully solved the N-Queens puzzle using AI search techniques.

References

1. Russell & Norvig – *Artificial Intelligence: A Modern Approach*
2. <https://www.n-queens.com>

Code:

```
import time
```

```
def solve_n_queens(n):
```

```
    def is_safe(board, row, col):
```

```
        for i in range(row):
```

```
            if board[i] == col or \
```

```
                board[i] - i == col - row or \
```

```
                board[i] + i == col + row:
```

```
            return False
```

```
    return True
```

```
def solve(board, row):
```

```
    if row == n:
```

```
        result.append(board[:])
```

```
    return
```

```
    for col in range(n):
```

```
        if is_safe(board, row, col):
```

```
            board[row] = col
```

```
            solve(board, row + 1)
```

```
            board[row] = -1
```

```
result = []
```

```
solve([-1] * n, 0)
```

```
return result
```

```
def print_solution(solution):
```

```
    n = len(solution)
```

```
    print("\nSolution board:")
```

```
    for row in range(n):
```

```
        print(' '.join('Q' if solution[row] == col else '.' for col in range(n)))
```

```
def animate_solution(solution):
```

```
    n = len(solution)
```

```
    board = [['.' for _ in range(n)] for _ in range(n)]
```

```

print("\nAnimating solution row by row:\n")
for row in range(n):
    board[row][solution[row]] = 'Q'
    for line in board:
        print(' '.join(line))
    print()
    time.sleep(0.4)

n = int(input("Enter the value of N (e.g., 12): "))
solutions = solve_n_queens(n)
print(f"\nTotal solutions for {n}-Queens: {len(solutions)}")
if solutions:
    print("First solution (column indices for each row):", solutions[0])
    if input("Show the first solution visually? (y/N): ").strip().lower() == 'y':
        print_solution(solutions[0])
    if input("Animate the first solution? (y/N): ").strip().lower() == 'y':
        animate_solution(solutions[0])

```

Output:

```

PS C:\Users\thand\OneDrive\Documents\AI&ES\co1at3pics> & C:\Users\thand\Ap
e c:/Users/thand/OneDrive/Documents/AIES/co1at3pics/n-queens.py
Enter the value of N (e.g., 12): 8

Total solutions for 8-Queens: 92
First solution (column indices for each row): [0, 4, 7, 5, 2, 6, 1, 3]
Show the first solution visually? (y/N): y

Solution board:
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .

```

3. Tic-Tac-Toe AI Challenge

Title

Tic-Tac-Toe AI Challenge

Game Platform

- **Platform:** Play Tic-Tac-Toe Online
- **Link:** <https://playtictactoe.org>

Objective

To defeat the AI opponent or achieve a draw by making optimal moves on a 3×3 game board.

Problem Statement

The player competes against an AI opponent by placing X or O on a 3×3 grid. The objective is to create a row, column, or diagonal of three identical symbols while preventing the opponent from doing the same.

AI Concept

- Adversarial Search
- Minimax Algorithm
- Alpha-Beta Pruning
- Game Tree Search

Initial and Goal State

Initial State: Empty 3×3 Tic-Tac-Toe board.

Goal State: Form three consecutive symbols horizontally, vertically, or diagonally before the opponent, or achieve a draw if no winning move exists.

Available Actions

- Place a symbol (X or O) in any empty square.

Solution Strategy

The AI uses the **Minimax algorithm** to evaluate all possible future game states and selects the move that maximizes its chances of winning while minimizing the opponent's chances. **Alpha-Beta Pruning** improves efficiency by eliminating branches that do not affect the final outcome.

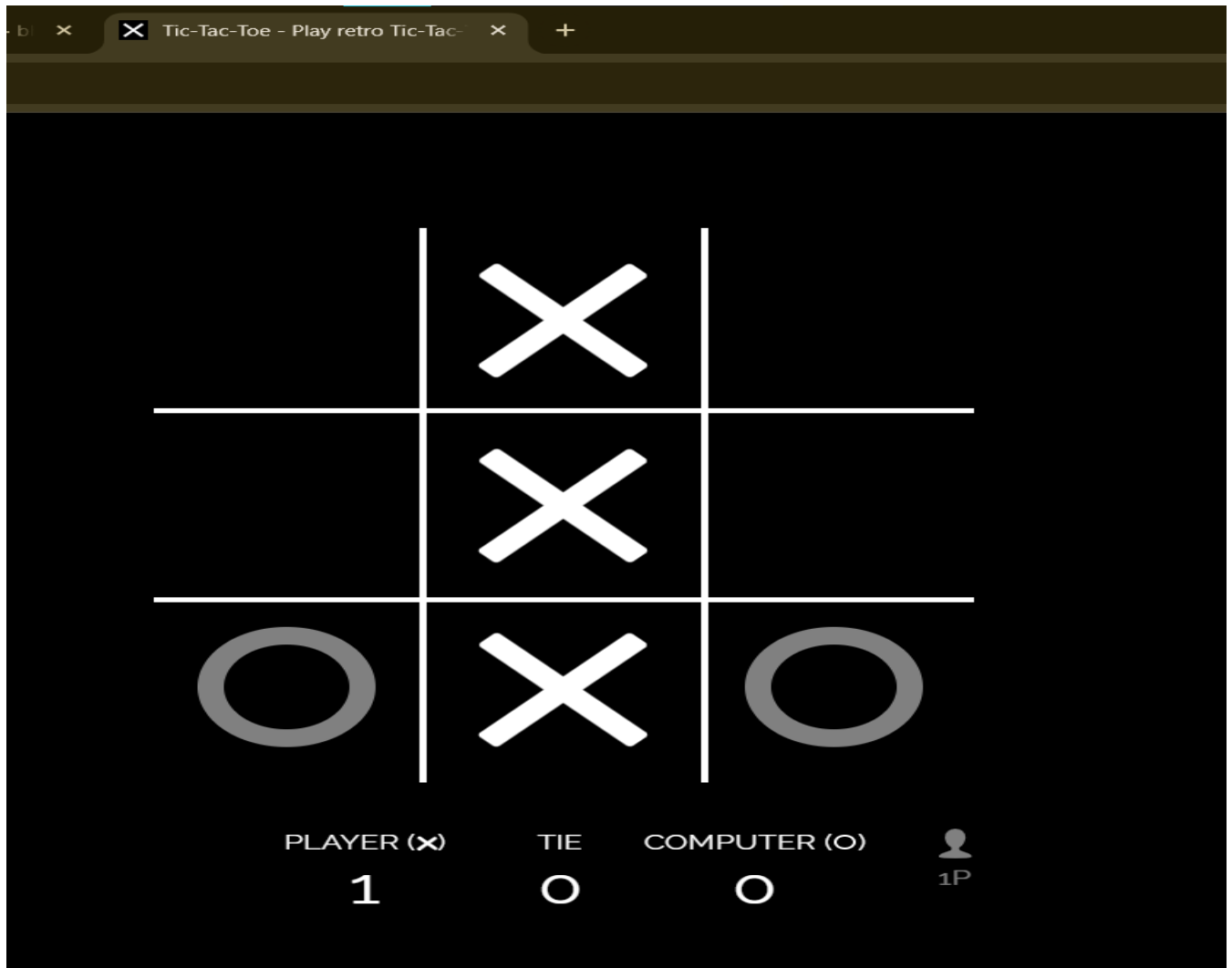
Steps

1. Begin by selecting the center or a corner square.
2. Block the opponent's immediate winning opportunities.
3. Create multiple winning possibilities.
4. Complete a row, column, or diagonal of three symbols.
5. If a win is not possible, force the game to end in a draw.

Performance

- Game completed successfully.
- **Result:** Win / Draw (*update according to your screenshot*).

Evidence



Efficiency

The AI selected optimal moves throughout the game, avoiding unnecessary mistakes and ensuring the best possible outcome.

Analysis

Tic-Tac-Toe is a classic example of an adversarial search problem where each player's decision affects the opponent's strategy. AI evaluates future moves before selecting the optimal action.

Alternative Solution

A Reinforcement Learning approach, such as **Q-Learning**, can enable the AI to learn optimal strategies through repeated gameplay instead of relying solely on Minimax.

Learning Outcome

This activity demonstrated how AI uses game-tree search, strategic planning, and decision-making algorithms in competitive environments.

Conclusion

The Tic-Tac-Toe AI Challenge provided practical experience with adversarial search and strategic decision-making, illustrating the effectiveness of the Minimax algorithm in game-playing AI.

References

1. Russell, S. J., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*.
2. Play Tic-Tac-Toe Online: <https://playtictactoe.org>

Code:

```
import random
```

```
BOARD_TEMPLATE = '''
```

```
{0} | {1} | {2}
```

```
---+---+---
```

```
{3} | {4} | {5}
```

```
---+---+---
```

```
{6} | {7} | {8}
```

```
'''
```

```
WINNING_LINES = [
```

```
(0, 1, 2),
```

```
(3, 4, 5),
```

```
(6, 7, 8),
```

```
(0, 3, 6),
```

```
(1, 4, 7),
```

```
(2, 5, 8),
```

```
(0, 4, 8),
```

```
(2, 4, 6)
```

```
]
```

```
def print_board(board):
```

```
    print(BOARD_TEMPLATE.format(*board))
```

```
def has_winner(board, mark):
```

```
    return any(all(board[pos] == mark for pos in line) for line in WINNING_LINES)
```

```
def is_board_full(board):
```

```
    return all(cell != ' ' for cell in board)
```

```
def get_player_move(board):
```

```
    while True:
```

```
        move = input('Enter your move (1-9): ').strip()
```

```
        if not move.isdigit():
```

```
            print('Please enter a number between 1 and 9.')
```

```
            continue
```

```

pos = int(move) - 1
if pos < 0 or pos > 8:
    print('Position must be within 1 and 9.')
elif board[pos] != ' ':
    print('That square is already taken.')
else:
    return pos

```

```

def get_computer_move(board, mark):
    opponent = 'X' if mark == 'O' else 'O'
    for pos in range(9):
        if board[pos] == ' ':
            board[pos] = mark
            if has_winner(board, mark):
                board[pos] = ' '
                return pos
            board[pos] = ' '
    for pos in range(9):
        if board[pos] == ' ':
            board[pos] = opponent
            if has_winner(board, opponent):
                board[pos] = ' '
                return pos
            board[pos] = ' '
    for pos in (4, 0, 2, 6, 8, 1, 3, 5, 7):
        if board[pos] == ' ':
            return pos

```

```

def play_game():
    board = [' '] * 9
    print('Welcome to Tic Tac Toe!')
    print('You are X, computer is O.')
    print_board(board)

```

```

while True:
    pos = get_player_move(board)
    board[pos] = 'X'
    print_board(board)
    if has_winner(board, 'X'):

```



```

        print('You win!')
        break
    if is_board_full(board):
        print('The game is a tie.')
        break

    print('Computer is making a move...')
    comp_pos = get_computer_move(board, 'O')
    board[comp_pos] = 'O'
    print_board(board)
    if has_winner(board, 'O'):
        print('Computer wins!')
        break
    if is_board_full(board):
        print('The game is a tie.')
        break

    if input('Play again? (y/N): ').strip().lower() == 'y':
        print('\n' * 2)
        play_game()

if __name__ == '__main__':
    play_game()

```

Output:

```

PS C:\Users\thand\OneDrive\Documents\AI\ES\co1at3pic>
e c:/Users/thand/OneDrive/Documents/AIES/co1at3pic
Enter your move (1-9): 9

  o | x | o
---+---+---
  x | x | o
---+---+---
  x | o | x

The game is a tie.
Play again? (y/N): 

```

4. Connect Four AI Challenge

Title

Connect Four AI Challenge (4 in A Line!)

Game Platform

- Platform: Maths is Fun
- Link: <https://www.mathsisfun.com/games/connect4.html>

Objective

To connect four pieces in a row before the opponent.

Problem Statement

Players must strategically place tokens while preventing the opponent from forming a line of four.

AI Concept

- Adversarial Search
- Minimax Algorithm
- Alpha-Beta Pruning

Initial and Goal State

Initial State: Empty Connect Four board.

Goal State: Four connected tokens of the same color.

Available Actions

- Drop a token into any one of the seven columns.

Solution Strategy

Minimax evaluates future game states while Alpha-Beta Pruning removes unnecessary branches to improve efficiency.

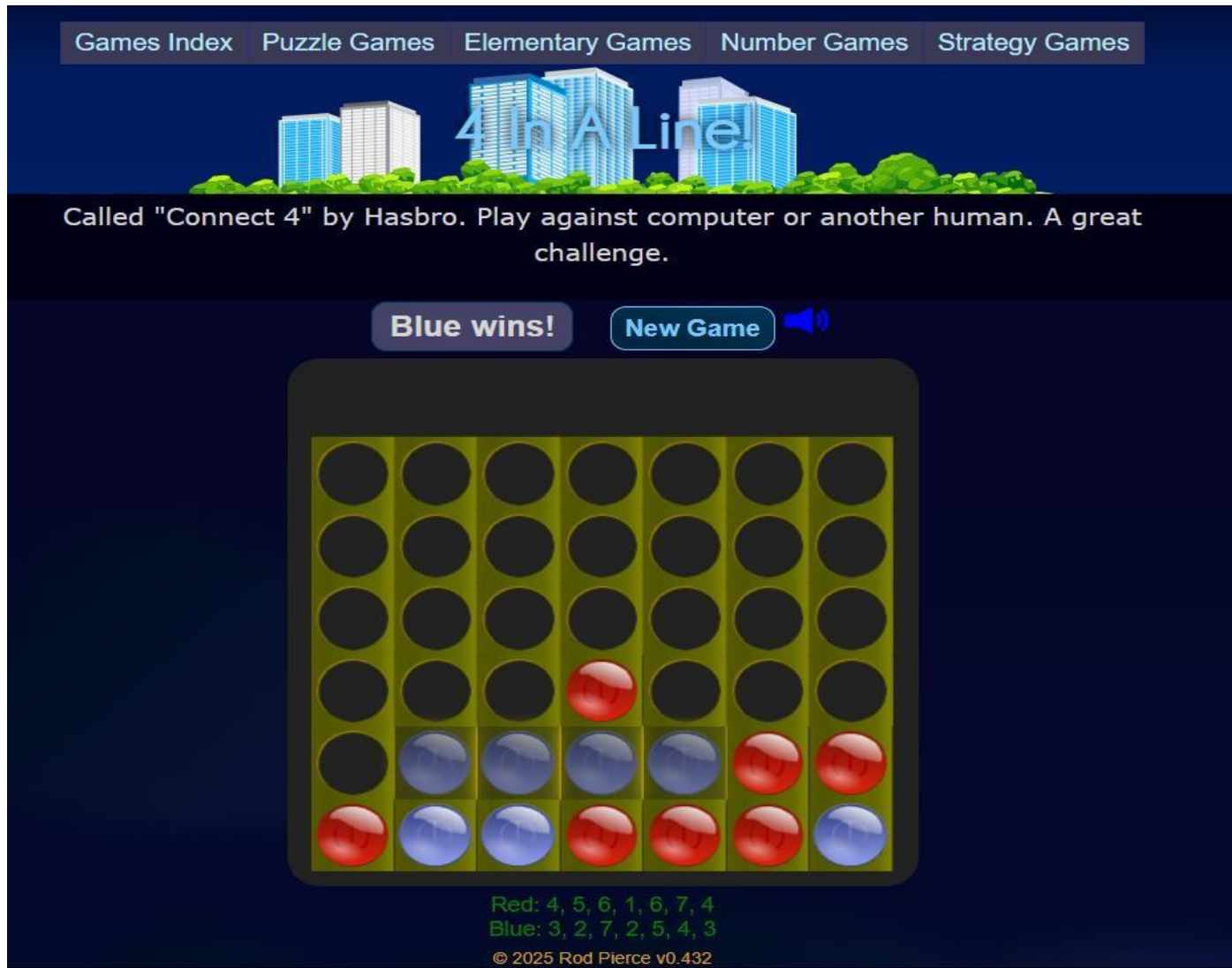
Steps

1. Control the center columns.
2. Build attacking positions.
3. Block opponent's winning chances.
4. Create a winning connection.

Performance

- Winner: Blue
- Winning Pattern: Diagonal

Evidence



Efficiency

The strategy successfully forced the opponent into defensive moves and secured victory.

Analysis

The activity demonstrates adversarial search, decision-making, and game-tree evaluation.

Alternative Solution

Reinforcement Learning (Q-Learning) can learn strategies through self-play instead of relying solely on Minimax.

Learning Outcome

Learned how AI predicts future moves and makes strategic decisions.

Conclusion

Successfully demonstrated game-playing AI using Minimax and Alpha-Beta Pruning.

References

1. Russell & Norvig – *Artificial Intelligence: A Modern Approach*
2. <https://www.mathsisfun.com/games/connect4.html>

Code:

```
import math
import time
```

```
def get_image_board():
    # 0 = Empty, 1 = Red, 2 = Blue
    return [
        [0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 0, 0, 0, 0],
        [0, 0, 0, 1, 0, 0, 0],
        [0, 2, 2, 2, 2, 1, 1],
        [1, 2, 2, 1, 1, 1, 2]
    ]
```

```
def check_winner(board, piece):
    # Check horizontal locations for win
    for c in range(4):
        for r in range(6):
            if board[r][c] == piece and board[r][c+1] == piece and board[r][c+2] == piece and
board[r][c+3] == piece:
                return True
    # Check vertical locations for win
    for c in range(7):
        for r in range(3):
            if board[r][c] == piece and board[r+1][c] == piece and board[r+2][c] == piece and
board[r+3][c] == piece:
                return True
    # Check positively sloped diagonals
    for c in range(4):
        for r in range(3):
            if board[r][c] == piece and board[r+1][c+1] == piece and board[r+2][c+2] == piece and
board[r+3][c+3] == piece:
                return True
    # Check negatively sloped diagonals
    for c in range(4):
        for r in range(3, 6):
            if board[r][c] == piece and board[r-1][c+1] == piece and board[r-2][c+2] == piece and
board[r-3][c+3] == piece:
                return True
    return False
```

```
def evaluate_window(window, piece):
    score = 0
    opp_piece = 1 if piece == 2 else 2
    if window.count(piece) == 4:
        score += 100
    elif window.count(piece) == 3 and window.count(0) == 1:
        score += 5
    elif window.count(piece) == 2 and window.count(0) == 2:
        score += 2
    if window.count(opp_piece) == 3 and window.count(0) == 1:
        score -= 4
    return score
```

```

def score_position(board, piece):
    score = 0
    center_array = [board[r][3] for r in range(6)]
    center_count = center_array.count(piece)
    score += center_count * 3
    for r in range(6):
        row_array = board[r]
        for c in range(4):
            window = row_array[c:c+4]
            score += evaluate_window(window, piece)
    for c in range(7):
        col_array = [board[r][c] for r in range(6)]
        for r in range(3):
            window = col_array[r:r+4]
            score += evaluate_window(window, piece)
    for r in range(3):
        for c in range(4):
            window = [board[r+i][c+i] for i in range(4)]
            score += evaluate_window(window, piece)
    for r in range(3):
        for c in range(4):
            window = [board[r+3-i][c+i] for i in range(4)]
            score += evaluate_window(window, piece)
    return score

def is_terminal_node(board):
    return check_winner(board, 1) or check_winner(board, 2) or len([c for c in range(7) if board[0][c] == 0]) == 0

def get_valid_locations(board):
    return [c for c in range(7) if board[0][c] == 0]

def get_next_open_row(board, col):
    for r in range(5, -1, -1):
        if board[r][col] == 0:
            return r

def minimax(board, depth, alpha, beta, maximizingPlayer):
    valid_locations = get_valid_locations(board)
    is_terminal = is_terminal_node(board)
    if depth == 0 or is_terminal:
        if is_terminal:
            if check_winner(board, 2):
                return (None, 1000000000000000)
            elif check_winner(board, 1):
                return (None, -1000000000000000)
            else:
                return (None, 0)
        else:
            return (None, score_position(board, 2))
    if maximizingPlayer:
        value = -math.inf
        best_col = valid_locations[0]
        for col in valid_locations:

```

```

        row = get_next_open_row(board, col)
        b_copy = [r[:] for r in board]
        b_copy[row][col] = 2
        new_score = minimax(b_copy, depth-1, alpha, beta, False)[1]
        if new_score > value:
            value = new_score
            best_col = col
        alpha = max(alpha, value)
        if alpha >= beta:
            break
    return best_col, value
else:
    value = math.inf
    best_col = valid_locations[0]
    for col in valid_locations:
        row = get_next_open_row(board, col)
        b_copy = [r[:] for r in board]
        b_copy[row][col] = 1
        new_score = minimax(b_copy, depth-1, alpha, beta, True)[1]
        if new_score < value:
            value = new_score
            best_col = col
        beta = min(beta, value)
        if alpha >= beta:
            break
    return best_col, value

def print_board(board):
    for row in board:
        print(" ".join(str(cell) for cell in row))

def animate_thinking():
    print("\nAI is thinking", end="")
    for _ in range(4):
        print('.', end="", flush=True)
        time.sleep(0.4)
    print("\n")

board = get_image_board()

print("Current Board State (0 = Empty, 1 = Red, 2 = Blue):")
print_board(board)

if check_winner(board, 1):
    print("\nResult: Red (1) wins!")
elif check_winner(board, 2):
    print("\nResult: Blue (2) wins!")
else:
    depth_input = input("\nEnter search depth for the AI (default 4): ")
    depth = int(depth_input) if depth_input.strip().isdigit() else 4
    print("\nCalculating optimal move...")
    best_move, best_score = minimax(board, depth, -math.inf, math.inf, True)
    animate_thinking()
    print(f"Optimal column to play for Blue (2): {best_move}, Score: {best_score}")

```

Output:

```
PS C:\Users\thand\OneDrive\Documents\AI\ES\co1at3pics> ^C
PS C:\Users\thand\OneDrive\Documents\AI\ES\co1at3pics> & C:\Users\thand\AppData\Local\Programs\Python\Python313\python.exe c:/Users/thand/OneDrive/Documents/AI\ES/co1at3pics/4_in_line.py
Current Board State (0 = Empty, 1 = Red, 2 = Blue):
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 1 0 0 0
0 2 2 2 2 1 1
1 2 2 1 1 1 2

Result: Blue (2) wins!
PS C:\Users\thand\OneDrive\Documents\AI\ES\co1at3pics> █
```

5. 8-Puzzle AI Challenge

Title

8-Puzzle AI Challenge (Sliding Puzzle)

Game Platform

- Platform: 8Puzzle.org
- Link: <https://8puzzle.org/>

Objective

To arrange numbered tiles in the correct order using the minimum number of moves.

Problem Statement

The player must transform a scrambled puzzle into the goal configuration using valid tile movements.

AI Concept

- State-Space Search
- Heuristic Search
- A* Search Algorithm

Initial and Goal State

Initial State: Scrambled 3×3 puzzle.

Goal State: Tiles arranged from 1 to 8 with the blank tile at the bottom-right.

Available Actions

- Move Blank Up
- Move Blank Down
- Move Blank Left
- Move Blank Right

Solution Strategy

A* Search uses heuristics such as Manhattan Distance to efficiently identify the optimal sequence of moves.

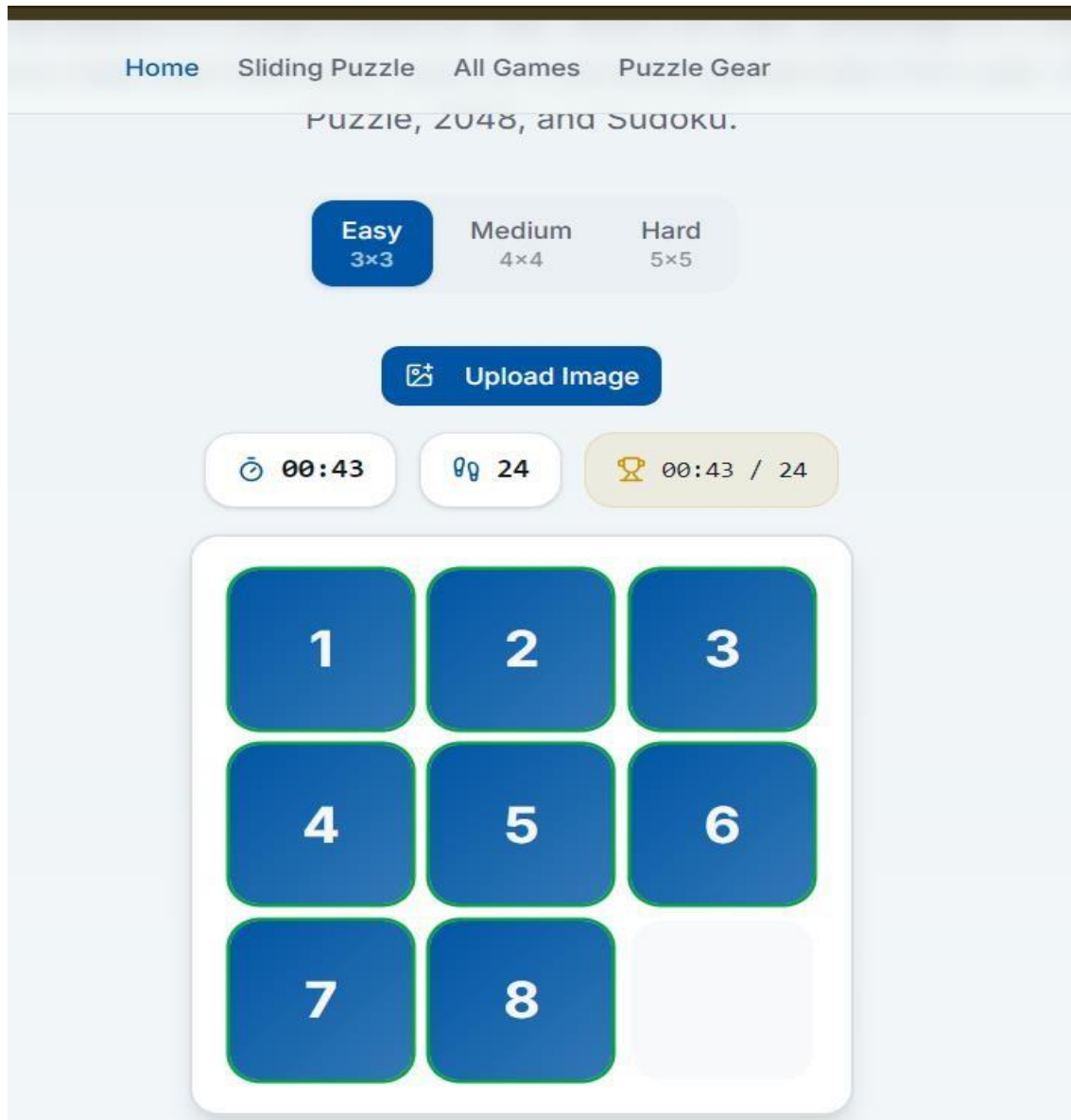
Steps

1. Arrange the first row.
2. Position the left column.
3. Solve the remaining tiles.
4. Place the blank tile in its final position.

Performance

- Difficulty: Easy (3×3)
- Time Taken: 43 seconds
- Total Moves: 24

Evidence



Efficiency

The puzzle was solved efficiently in 24 moves, demonstrating near-optimal performance.

Analysis

The activity highlights the importance of heuristic functions in reducing the search space and improving solution speed.

Alternative Solution

IDA* Search can efficiently solve larger sliding puzzles with lower memory usage.

Learning Outcome

Learned the importance of heuristic search techniques in solving complex state-space problems.

Conclusion

Successfully solved the puzzle using heuristic search principles.

References

1. Russell & Norvig – *Artificial Intelligence: A Modern Approach*
2. <https://8puzzle.org/>

Code:

```
import heapq
import time

class PuzzleState:
    def __init__(self, board, parent, moves, cost):
        self.board = board
        self.parent = parent
        self.moves = moves
        self.cost = cost

    def __lt__(self, other):
        return self.cost < other.cost

def manhattan_distance(board, goal):
    distance = 0
    for i in range(9):
        if board[i] != 0:
            target_idx = goal.index(board[i])
            distance += abs(i // 3 - target_idx // 3) + abs(i % 3 - target_idx % 3)
    return distance

def solve_8_puzzle(start, goal):
    start_state = PuzzleState(start, None, 0, manhattan_distance(start, goal))
    pq = [start_state]
    visited = set([tuple(start)])

    while pq:
        current = heapq.heappop(pq)

        if current.board == goal:
            path = []
            while current:
                path.append(current.board)
                current = current.parent
            return path[::-1]
```

```

zero_idx = current.board.index(0)
r, c = zero_idx // 3, zero_idx % 3

directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
for dr, dc in directions:
    nr, nc = r + dr, c + dc
    if 0 <= nr < 3 and 0 <= nc < 3:
        new_idx = nr * 3 + nc
        new_board = list(current.board)
        new_board[zero_idx], new_board[new_idx] = new_board[new_idx],
new_board[zero_idx]

        if tuple(new_board) not in visited:
            visited.add(tuple(new_board))
            cost = current.moves + 1 + manhattan_distance(new_board, goal)
            heapq.heappush(pq, PuzzleState(new_board, current, current.moves + 1, cost))
return None

def print_board(board):
    for row in range(3):
        print(' '.join(str(board[row * 3 + col]) for col in range(3)))
    print()
print("Enter the start state (9 numbers separated by space, use 0 for empty space):")
start_board = list(map(int, input().split()))

print("Enter the goal state (9 numbers separated by space, use 0 for empty space):")
goal_board = list(map(int, input().split()))

solution = solve_8_puzzle(start_board, goal_board)
if solution:
    print(f"\nSolved in {len(solution) - 1} moves.")
    if input("Show each move step by step? (y/N): ").strip().lower() == 'y':
        for move_index, board in enumerate(solution):
            print(f"Move {move_index}:")
            print_board(board)
            time.sleep(0.5)
else:
    print("No solution found.")

```

Output:

```
PS C:\Users\thand\OneDrive\Documents\AIIES\co1at3pics> & C:\Users\thand\OneDrive\Documents\AIIES\co1at3pics\8-puzzle.exe c:/Users/thand/OneDrive/Documents/AIES/co1at3pics/8-puzzle.txt
Enter the start state (9 numbers separated by space, use 0 for blank)
1 2 3 4 5 6 7 0 8
Enter the goal state (9 numbers separated by space, use 0 for blank)
1 2 3 4 5 6 7 8 0

Solved in 1 moves.
Show each move step by step? (y/N): ☐
```

Structure of the Report – Game-Based Learning

S.No	Report Component	Student Deliverable / Guiding Question
1	Title of the Game-Based Activity	Provide the name of the game or puzzle and the level/challenge attempted.
2	Game Platform / Link	Mention the platform or website used and provide the corresponding game link.
3	Objective of the Activity	Explain the objective of the game and the target that needs to be achieved.
4	Problem Statement	Describe the problem or challenge given in the game. What is the student required to solve?
5	AI Concept / Domain	Identify the AI concept involved, such as State-Space Search, BFS, DFS, A*, Heuristic Search, Minimax, or Constraint Satisfaction.
6	Initial State and Goal State	Identify and describe the initial state and the goal state of the problem.
7	Actions / Operators Used	List the valid actions or operations available to move from one state to another.
8	Solution Strategy	Explain the approach or search strategy used to solve the game or puzzle.
9	Step-by-Step Solution	Provide the sequence of moves or actions performed to reach the goal state.
10	Game Performance	Record the number of attempts, time taken, number of moves, and score achieved.
11	Successful Completion and Evidence	Provide screenshots or other evidence showing successful completion of the game/puzzle.
12	Efficiency and Optimality	Analyse whether the solution was optimal or efficient. Can the puzzle be solved using fewer moves or less time?
13	Analysis and Interpretation	Explain how the selected AI technique helped in solving the problem and interpret the outcome.
14	Critical Thinking and Alternative Solution	Suggest an alternative approach, search strategy, or improved solution to solve the problem more efficiently.
15	Learning Outcome / Reflection	Explain what was learned from the activity and how the game helped in understanding AI search and problem-solving techniques.
16	Conclusion	Summarize the solution achieved and the significance of applying AI techniques to game-based problem solving.
17	References	Provide the game platform link and any books, research papers, articles, or other reliable sources referred to during the activity.

Evaluation Rubrics

Criteria	Percentage (%)	Excellent	Good	Satisfactory	Needs Improvement
Understanding of the Problem / Game Objective	10%	Clearly understands the problem and game objective	Understands the objective with minor errors	Partially understands the problem	Does not understand the objective
Application of AI Search / Problem-Solving Technique	15%	Correctly applies the appropriate AI search or problem-solving technique	Applies the technique with minor errors	Shows limited application of the technique	Unable to apply the appropriate technique
Successful Completion of the Game / Puzzle	20%	Successfully solves/completes the game independently	Completes the game with minimal assistance	Partially completes the challenge	Unable to complete the challenge
Efficiency of Solution / Optimal Moves	10%	Solves using optimal or highly efficient moves	Uses a reasonably efficient sequence of moves	Uses several unnecessary moves	Solution is highly inefficient or incomplete
Documentation / Evidence of Completion	15%	Provides clear steps, score, screenshots, and evidence of completion	Provides most required evidence	Provides incomplete steps or evidence	Provides no proper documentation or evidence
Analysis and Interpretation of the Solution	15%	Clearly analyses the solution and explains the AI technique and outcome	Provides a good analysis with minor gaps	Provides limited analysis or explanation	Unable to analyse or interpret the solution
Originality and Critical Thinking	15%	Demonstrates excellent independent thinking and explores an effective or alternative solution	Demonstrates good independent thinking	Shows limited critical thinking	Shows little or no independent thinking